Graphics Programming

code

```cpp
#include <GL/glew.h>
#include <GL/freeglut.h>
#include <iostream>
#include <cstdlib>
#include <ctime>

float squareX = 50.0f;
float squareY = 50.0f;
float squareSize = 60.0f;
float speed = 4.0f;
const int WIDTH = 800;
const int HEIGHT = 500;
bool keys[256];
float squareColor[3] = { 0.0f, 1.0f, 1.0f };

void keyDown(unsigned char key, int x, int y)
{
    keys[key] = true;
    if (key == ' ')
    {
        squareColor[0] = static_cast<float>(rand()) / RAND_MAX;
        squareColor[1] = static_cast<float>(rand()) / RAND_MAX;
        squareColor[2] = static_cast<float>(rand()) / RAND_MAX;
    }
}

void keyUp(unsigned char key, int x, int y) { keys[key] = false; }

void specialKeyDown(int key, int x, int y)
{
    switch (key)
    {
    case GLUT_KEY_UP: keys['U'] = true; break;
    case GLUT_KEY_DOWN: keys['D'] = true; break;
    case GLUT_KEY_LEFT: keys['L'] = true; break;
    case GLUT_KEY_RIGHT: keys['R'] = true; break;
    }
```

Data Sources

moving square.cpp

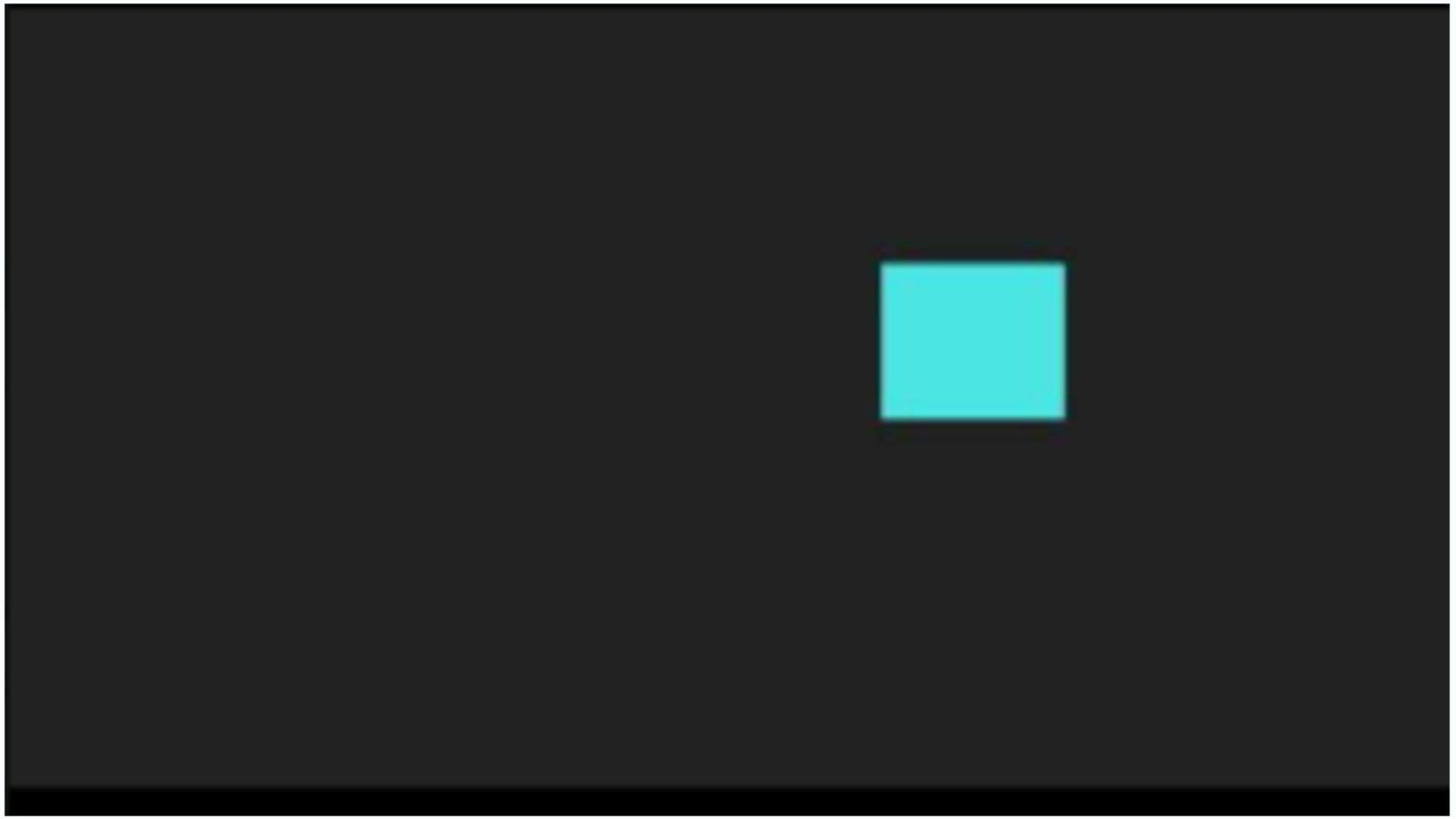
Miscellaneous Files

(Global Scope)

```
37          }
38      }
39
40      void specialKeyUp(int key, int x, int y)
41      {
42          switch (key)
43          {
44          case GLUT_KEY_UP: keys['U'] = false; break;
45          case GLUT_KEY_DOWN: keys['D'] = false; break;
46          case GLUT_KEY_LEFT: keys['L'] = false; break;
47          case GLUT_KEY_RIGHT: keys['R'] = false; break;
48          }
49      }
50
51      void clampPosition()
52      {
53          if (squareX < 0) squareX = 0;
54          if (squareY < 0) squareY = 0;
55          if (squareX + squareSize > WIDTH) squareX = WIDTH - squareSize;
56          if (squareY + squareSize > HEIGHT) squareY = HEIGHT - squareSize;
57      }
58
59      void update()
60      {
61          if (keys['U']) squareY -= speed;
62          if (keys['D']) squareY += speed;
63          if (keys['L']) squareX -= speed;
64          if (keys['R']) squareX += speed;
65          clampPosition();
66      }
67
68      void drawSquare()
69      {
70          glBegin(GL_QUADS);
71          glColor3f(squareColor[0], squareColor[1], squareColor[2]);
72          glVertex2f(squareX, squareY);
73          glVertex2f(squareX + squareSize, squareY);
74          glVertex2f(squareX + squareSize, squareY + squareSize);
```

```
75          glVertex2f(squareX, squareY + squareSize);
76          glEnd();
77      }
78
79      void display()
80      {
81          update();
82          glClear(GL_COLOR_BUFFER_BIT);
83          drawSquare();
84          glutSwapBuffers();
85      }
86
87      void timer(int = 0)
88      {
89          glutPostRedisplay();
90          glutTimerFunc(16, timer, 0);
91      }
92
93      int main(int argc, char** argv)
94      {
95          srand(static_cast<unsigned int>(time(0)));
96          glutInit(&argc, argv);
97          glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
98          glutInitWindowSize(WIDTH, HEIGHT);
99          glutCreateWindow("Moving Square - OpenGL");
100         GLenum err = glewInit();
101         if (err != GLEW_OK) { std::cerr << "GLEW init failed\n"; return -1; }
102         glMatrixMode(GL_PROJECTION);
103         glLoadIdentity();
104         glOrtho(0, WIDTH, HEIGHT, 0, -1, 1);
105         glMatrixMode(GL_MODELVIEW);
106         glLoadIdentity();
107         glClearColor(0.133f, 0.133f, 0.133f, 1.0f);
108         glutDisplayFunc(display);
109         glutKeyboardFunc(keyDown);
```

Run the code

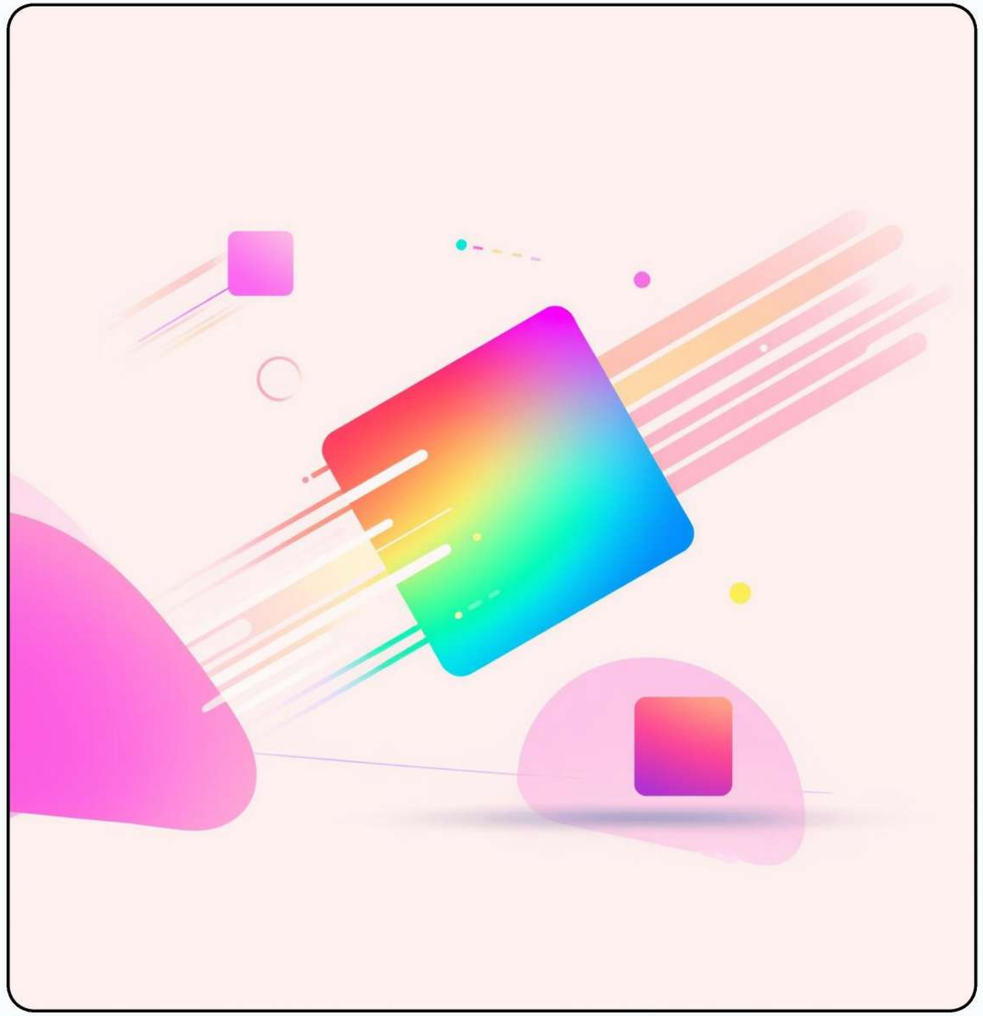


Introduction to 2D Graphics Programming

Exploring OpenGL and FreeGLUT Basics

This project introduces the fundamentals of **2D graphics programming**, focusing on creating an interactive application featuring a movable square that changes color. Engage with the basics of graphics!

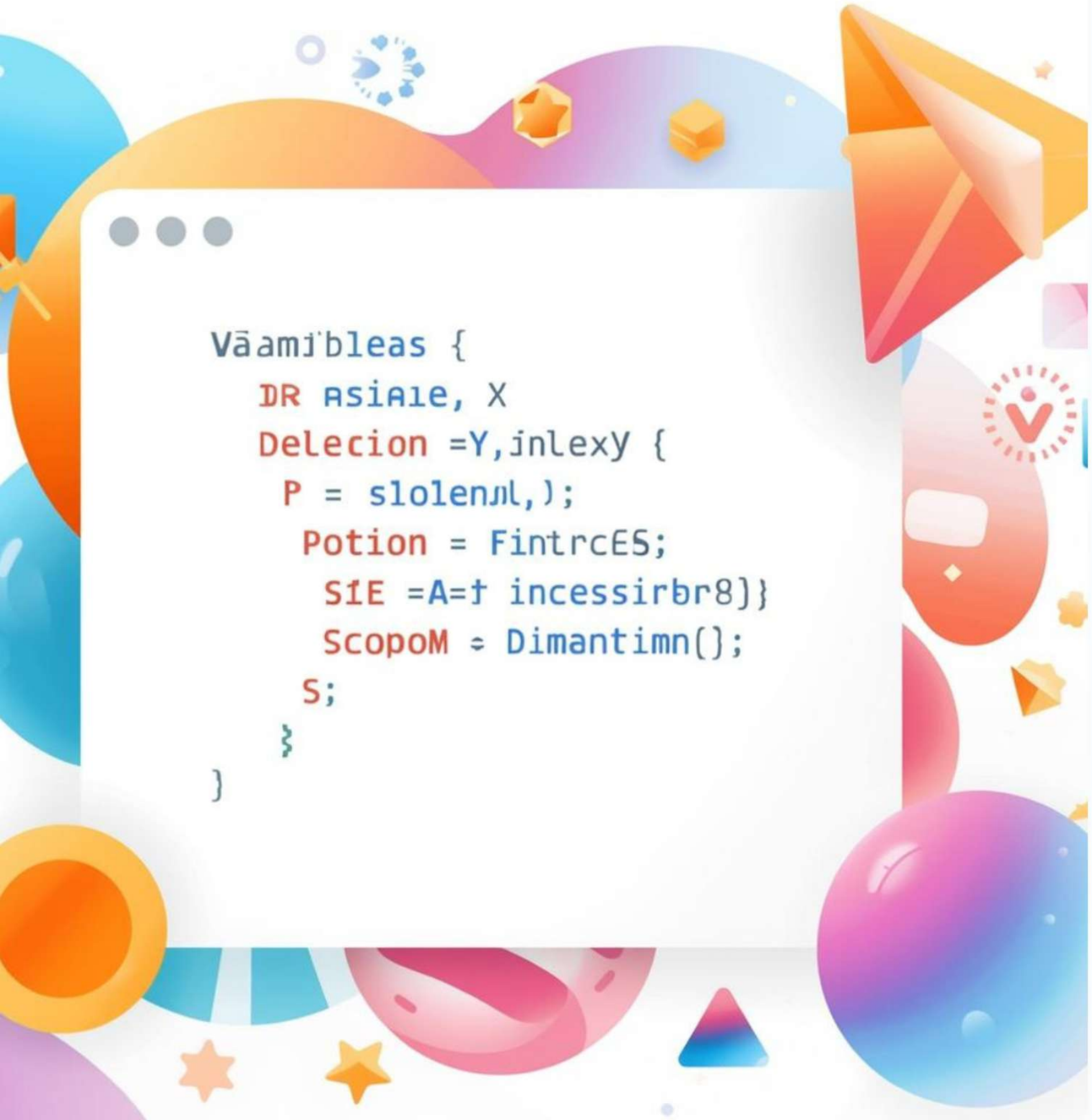
Moving the Square



Core Variables for the Square Setup

Defining Movement and Appearance Parameters

- Square position (X, Y coordinates)
- Square size and movement speed
- Square color (RGB values)
- Window width and height
- Background color configuration



Defining Square Properties

Square Positioning Explained

X and Y coordinates

Movement Speed Insights

How fast it moves

Color Representation Details

RGB value usage

Project Features

Movement

Smooth, fluid square movement

Color

Real-time color changes occur

Input

Robust handling of multiple keys

Code

Beginner-friendly structure for extensibility

Keyboard Input Management

Normal Keys

Space key changes the square's color.

Special Keys

Arrow keys move the square around screen.

Smooth Movement

Ensures consistent control and user feedback.

Key State Array

Tracks continuous key presses for smooth action.

Input Callbacks

Update key states for real-time interaction.

Drawing the Square with OpenGL

Vertices

Define four corners to draw the square.

Color

Specify RGB values for the square's appearance.

Primitives

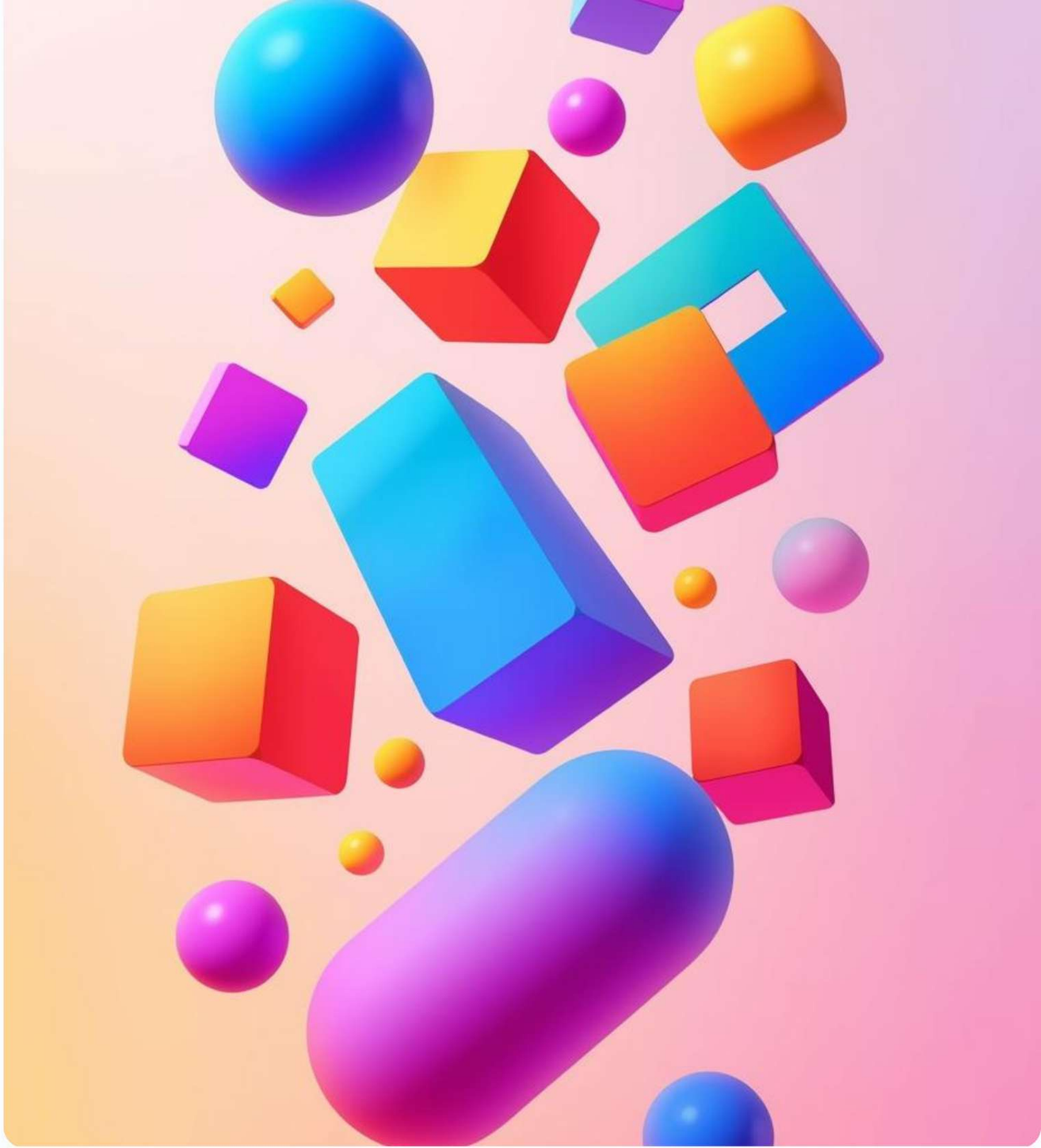
Use glBegin(GL_QUADS) to start drawing.

Animation Loop

Updating the Display

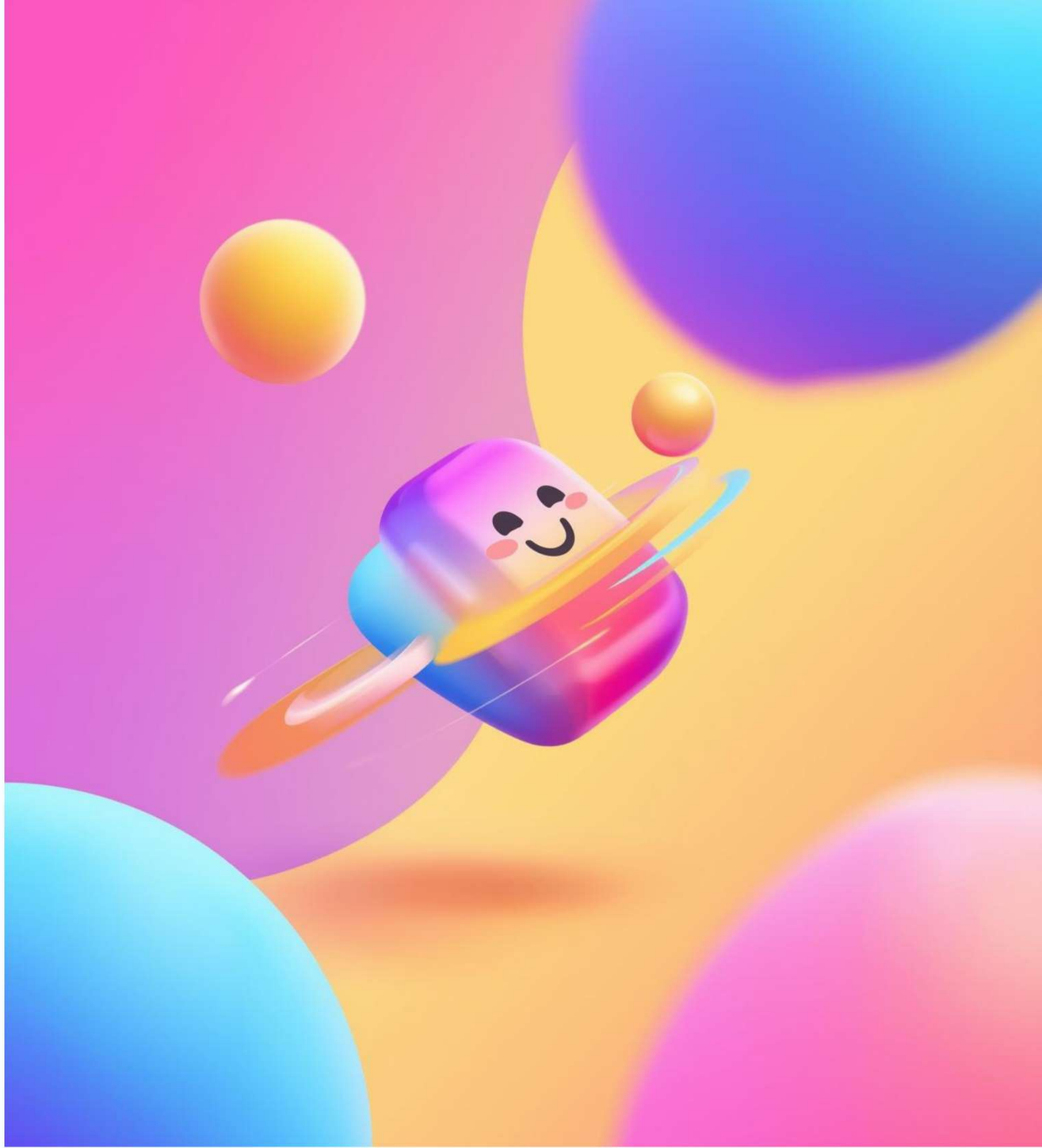
Interacting with the Colorful Square

Users will engage with a vibrant square on the screen. Arrow keys allow movement, while the Space key triggers real-time color changes, making the application lively and interactive for users.



Rendering and Animating the Square

The drawing process utilizes OpenGL primitives to create the square, while the animation loop ensures smooth movement and color changes via continuous input detection and timer-based updates at ~60 FPS.



Key Features of the Project

Smooth Square Movement

Continuous movement with controls

Real-Time Color Changes

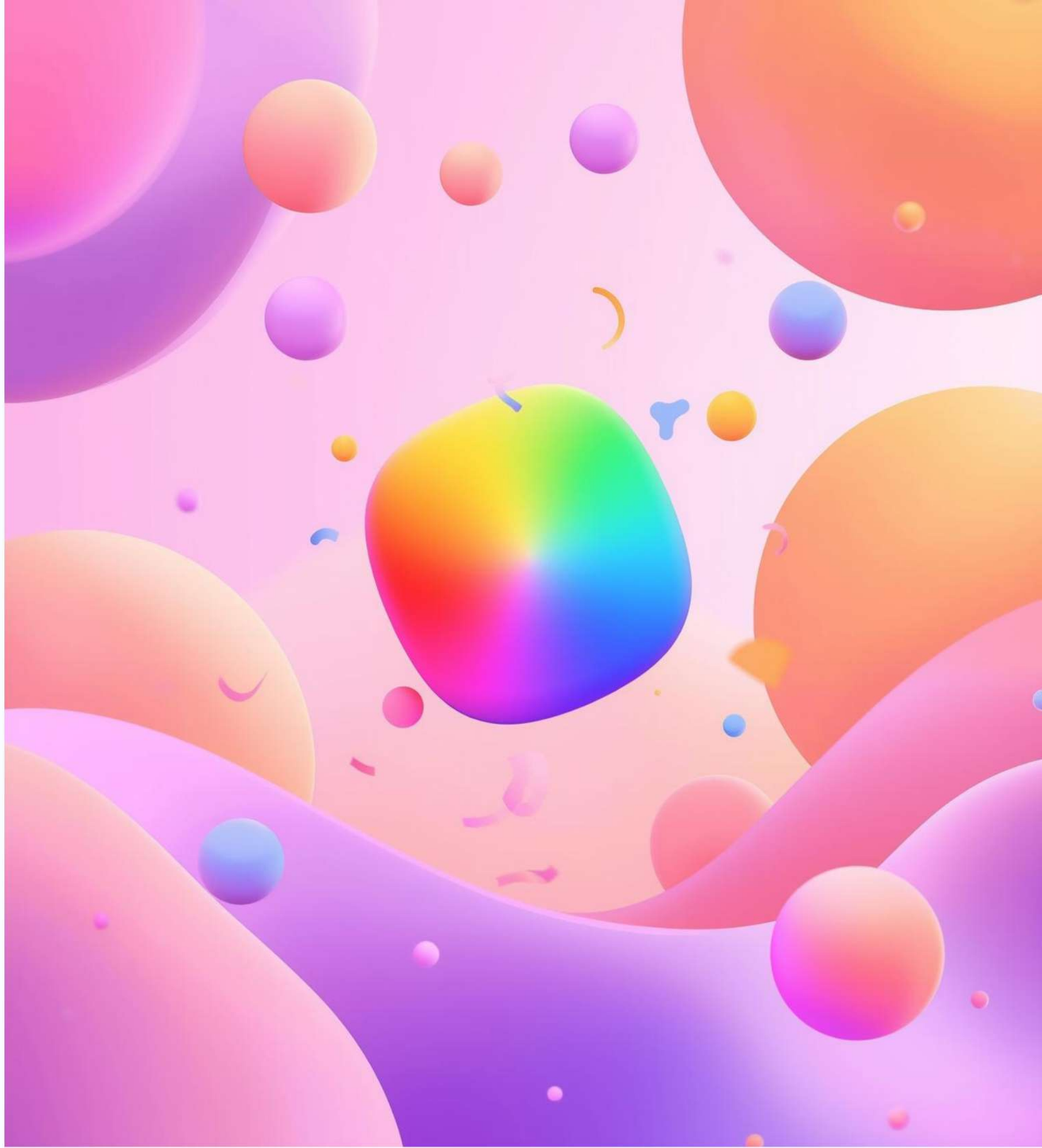
Instant feedback on interaction

Robust Input Handling

Multiple key detection capability

Key Features of the Project

providing a fun way to understand 2D graphics programming fundamentals with OpenGL and FreeGLUT.



Recap and Next Steps in Graphics

In summary, we've explored essential **2D graphics programming** concepts, including window creation, input handling, and animation. These fundamentals pave the way for further exploration of OpenGL's exciting capabilities

Thank You


gluBeins GL_QUAD
pc mertatlmbe
pertrins GL_QUAD
}
vertices GL_QUADS =>